Emergence of navigation behavior and spatial representation in agents with theta sequences based intrinsic reward

Author: Jannek Schaffert
Supervision: Xiao-Xiong Lin, Christian Leibold

Abstract

A number of attempts have been undertaken to train deep reinforcement learning model without the need for external reward. Here we suggest an intrinsic reward paradigm that leads to both emergence of spatial representations, as well as exploratory behaviour. This requires disentangling the contributions of neural encoding mechanisms from the agents' exploratory behaviour. For encoder-decoder separated, internally-based rewards, we evaluated several training regimes, all converging on non-trivial solutions. Across all conditions, agents outperformed a random baseline, yet a pure punishment paradigm for encoder learning consistently yielded the most precise place fields. Punished agents explored a larger fraction of the arena per episode, suggesting that broader behavioural sampling facilitates richer neural maps. Conversely, agents trained with alternative objectives exhibited circular trajectories that over-represented limited regions. Sparsification of inputs via constant punishment reduced overall activity, sharpening landmark selectivity, but introduced noisy, fragmented activations that were especially pronounced in the punished condition. Adjusting the intercept to raise activity levels did not alter these patterns or the resulting behavior. While auxiliary losses improved batch-wise performance, their impact on core place-field quality varied. Notably, encouraging inactive sequences over a batch amplified the number of active sequences, thereby compressing each sequence's spatial footprint and enhancing tuning precision. Overall, our findings highlight the intertwined roles of exploration strategy and loss-function design in shaping emergent place fields, and underscore the need for systematic probing of individual loss contributions.

Contents

1 Introduction

The hippocampus plays a crucial role in spatial navigation and episodic memory, as damage impairs memory formation in humans [13] and place cells tuned to specific locations are present in the hippocampus of rodents [14] and humans [4]. This suggests the idea of spatial learning through the formation of episodic memories [12]. This idea is reinforced by the observations of hippocampal replay sequences related to planning [7, 5]. In previous work of the supervisors, the idea of utilizing a reservoir of long hippocampal sequences (analog to cornu ammonis 3 (CA3)) with sparse input (from the dentate gyrus (DG)) was proposed to generate representations, which allow for successful learning of simple navigational tasks [8] and implemented in a deep Reinforcement Learning paradigm [9].
Here we want to extend this framework to generate these representations without the need for a task relying on external reward. The idea is to more closely model biological exploration and behaviour, where external reward is not always present. Additionally in the emergence of place fields, these tend to over-represent space near a goal location, leading to a non-uniform representation of space. We utilize theta-locking in sequences as a step marker, resulting in an internal metric representing space through time.

1.1 Motivation

A number of attempts for training an agent without the need for external reward have been proposed. Moreno-Bote and Ramírez-Ruiz [11] compare three such approaches, shortly outlined here:
Empowerment [6] calculates the maximum mutual information (channel capacity) between the next n-actions and the resulting state s_{t+n}. For computational purposes the horizon is chosen to be relatively small, limiting the capabilities of long-term planning. This leads to an exploration of states, which predictably lead to many future accessible states.
The Free Energy Principle [3] tries to minimize the expected free energy. The objective is to find the optimal 1-step policy to minimize the expected KL divergence between a generated distribution of how desirable states are (allowing the survival of the agent) and a time-independent target distribution. Behaviour heavily depends on the target states, finding a policy, which increases the probability of visiting these states.
Maximum Occupancy Principle [16] maximizes the cumulative future weighted action-state transition entropy. Behaviour thus tries to visit states through actions with low visit probability, while striving for actions leading to states with many more actions in order to not get stuck, which would lead to a decrease in entropy.
All of these approaches utilize some form of optimization over action-state space, not relying on completely internal metrics. Additionally, one key issue in all reward-free paradigms is the lack of a general evaluation metric and the definition of a desired behaviour. In Moreno-Bote and Ramírez-Ruiz [11] the main evaluation is looking at the behaviour of agents and evaluating this in terms of the question 'Which reward-free objective leads to behaviours that are more complex, rich and interesting?'[11].
While this is generally possible in our model as well (e.g. in terms of occupancy of the environment), we are also interested in what representations of space emerge from training in the internal activity of an agent. Ideally some form of place fields emerge, which might be used for downstream transfer learning of other spatial navigation tasks with actual external reward. However the sole emergence of place fields should not give rise to under-complex behaviour, which could severely limit the exploration of the entire environment, resulting in a skewed representation of space for locations with higher occupancy. Additionally if considering this as pre-training for other tasks, a general exploration of state-space is seen as beneficial.
Combining both place field generation and action generation to play off of each other would ideally result in place fields far apart from each other to cover the entire environment and actions that traverse the space between place fields on a direct path. To achieve this, there is a general need to balance learning of DG activity and action generation, which we will explore more in section 2.3 after introducing the architecture and model structure of our agent.

2 Methods

2.1 Architecture

The implementation of the framework used for training and evaluation is based on the SampleFactory [15] architecture, with custom modifications. It uses an asynchronous Actor-Critic-Architecture [10] with proximal policy optimization [19] as well as General Advantage Estimation [18]. Additionally there is entropy regularization added for all training runs.
The custom additions implement the means to arbitrarily inject a Learner class without interference with the rest of the codebase as well as minor adjustments to the overall architecture described in more detail in the Appendix.
The agent navigates in a virtual environment (fig. 1, Panel D), generated using Deepmind Lab [1]. It simulates a continuous, 3D virtual environment, consisting of 19×19 squares with sparse obstacles (15% of the tiles) randomly placed. The layout of the walls is kept fixed, for individual trials, unless otherwise stated. The walls have continuous visual textures and the first-person (egocentric) view is generated at 60 fps at half the wall height with a field of view of 90°. The resolution is 92×76 pixels in RGB channels.
In previous works of the supervisors the agent is placed randomly at least five units from a goal location. On successfully reaching the location, a reward is given and the trial ends. Otherwise a trial is ended after a maximum of 7200 frames. This setup will be referred to here as transfer-learning.
In the pre-training, performed only for this project, there is no external reward, which gets instead replaced by an engineered internal reward. The layout is the same as for transfer-learning and the trial length is kept fixed at 7200 frames. All other parameters concerning the general architecture are kept the same, except that starting positions now include the entire map, since there is no goal location anymore. The experiments were run on the NEMO computation cluster, either on the genoa or milan partition. These partitions are pure CPU partitions. In general the entire project would run on GPU nodes as well, with only little modification needed.
After the training phase of each trial, a separate phase of data collection is performed, which runs for 50000 environment frames. The data is then post-processed and analyzed. Training statistics are collected through wandb.ai[2]

2.2 Model structure

The entire structure of the model is displayed in fig. 1 and explained in the following:
The visual pipeline is intended to mimic biological cortical vision preprocessing, generating generic visual features for the hippocampus from egocentric visual input. We pre-trained a ResNet Encoder and kept it fixed during all trials.
Since the main cortical input into the hippocampus comes through the Dentate Gyrus (DG), the pre-processed visual features are projected through a fully connected linear layer followed by an additional batch normalization (retention rate: 0.9 per batch) and a high activation threshold. This ensures sparsity of activation as seen in active DG granule cells in a given environment. The sparseness can be controlled by the intercept factor (Threshold τ in fig. 1), defining how many rolling standard deviations a signal must surpass in order to activate a sequence. The estimation of the standard deviation is

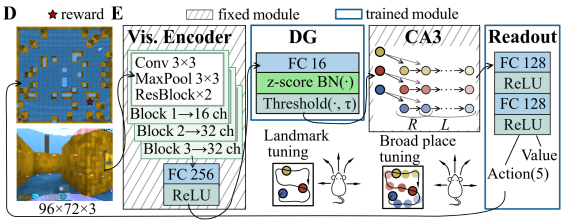

Figure 1: Model & environment structure: **D** Top: Layout of the map. Reward location is only active in external reward tasks. Bottom: Egocentric visual input for the agent. **E**: Visual structure of the agent from visual input to action/value generation. See text on the left for a detailed description. Image adapted with permission from Xiao-Xiong Lin.

performed on a per-neuron basis.
CA3 is modelled as a fixed RNN shift register, propagating incoming activity along pre-wired sequences of length l. With the number of consecutive theta cycles L and the number of active units per cycle R, we get the relation $l = L + R - 1$.
The entire CA3-activity is fed into a decoder with two linear, fully-connected layers and a ReLu-activation. Actions and values are generated as two separate linear read-out layers from the decoder output.

2.3 Mathematical properties

With the general idea of pre-training the model using optimization of some internal metric, there are a couple of different approaches to generate this metric from the described model structure in the last section 2.2. A general feature of each sequence f at every time step t_{now} is the time of last input $\delta t^{last}_{f,input} = t_{now} - t^{last}_{f,input}$. For a set of sequences $\mathcal{F}$ this gives us a vector $P_t = \{\delta t^{last}_{f,input}, f \in \mathcal{F}\}$, denoting the progression of how far the last input was propagated along a sequence. From this we can calculate a distance matrix D as the difference in time between the last activation of a pair of sequences:

$$D_t = abs(P_t^\top - P_t). \tag{1}$$

By definition, this matrix is symmetrical with all diagonal elements as zero. Naively one could average over all entries in D_t to get a value, which can be used for optimization. However most entries in D_t represent values only changed in the past and not affected by the corresponding action, minimizing the influence of the action on reward, which could lead to unstable training. Therefore the notion of an activated sequence is introduced: A sequence f is considered activated at time step t, if it gets input from the DG projection and did not receive any input in the last R time steps, where R is again the number of active units per cycle. This definition was chosen in order to allow for consecutive input for sequences, with little time in between, where we cannot expect the visual input to change much.
We can therefore take a row i of D_t, representing the difference in progression for an activated sequence i to all other sequences. Taking the minimum r of this row gives us the time between last input to another sequence, which we will take as a basis for defining the intrinsic reward module in section 3.

3 Intrinsic Reward Module

The general architecture behind deep reinforcement learning models uses an actor-critic structure. While the actor purely focuses on action generation from observations, the critic constantly estimates the total expected future reward. This estimation is used to update the parameters of the actor, promoting actions, which lead to a higher estimated reward. The critic is trained on the actually received reward. While normally this reward is obtained through the environment, here we explore the possibility of coupling the reward to an internal metric r, calculable from the structure of the agent (as seen in section 2.3). In this report we refer to a

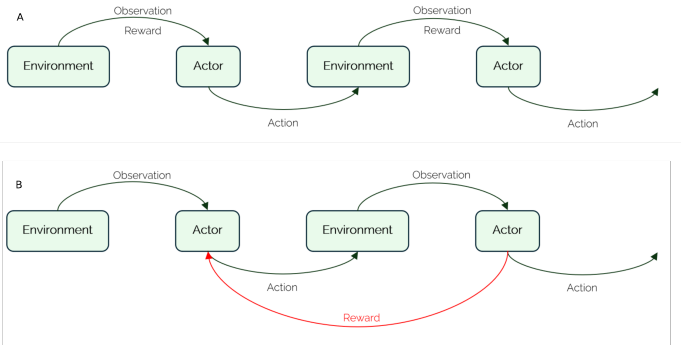

Figure 2: **A**: The original structure used in an actor-critic architecture. Here the sequence of observation & reward gathering to produce a subsequent action is shown. **B**: For internal reward training this structure needs adjustment, especially for the decoder. To couple the reward to a specific action, it is calculated from the sequence core one time step later.

positive reward as an encouragement,
a negative reward is dubbed punishment.
In order to balance the emergence of place fields and exploratory behaviour, the DG projection (from now on referred to as encoder) and decoder need to be trained differently. In an ideal case we could train the encoder on maximizing r, in order to push place fields further apart and cover the entire environment. In contrast the decoder would be trained on minimizing r which in theory leads to a selection of actions, which take the shortest path between place fields, resulting in the exploration of the entire environment. This creates a pull-push mechanism in the learning between encoder and decoder, improving both parts of the model at the same time. Note that there is a shift in the assignment of reward for the decoder (see fig. 2). This is due to the fact that the metric r, has to be calculated after action generation, to ensure temporal effectiveness.
In practice we utilize the existing actor-critic architecture for the decoder training. Since this tries to maximize reward, we calculate the custom decoder reward as $r_{dec} = -r + b_{dec}$, where $b_{dec} = L + R - 1$, to ensure a strictly positive reward. For the encoder we evaluate three different approaches all relying on $r_{enc} = r - b_{enc}$ with different b_{enc}, keeping the pull-push mechanism between encoder and decoder.
Firstly there is $b_{enc} = 0$, giving encouragement to the encoder as well. Effectively this should increase the overall activity of the encoder by only ever encouraging activation of feature nodes. The opposite is $b_{enc} = L + R - 1$, resulting in a punishment, which is expected to decrease activity levels. As a third option there is $b_{enc} = \overline{r}$ as the mean of r over a mini-batch. Thus there is encouragement as well as punishment, ideally keeping the overall activity levels the same.
In the actor-critic architecture the loss is calculated from an advantage (cumulative, discounted future reward) $A_t = Q_t(s_t, a_t) - V(s_t)$:

$$L_{dec}^{CLIP} = \mathbb{E}\left[min\left(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right], \qquad r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta,old}(a_t|s_t)}, \tag{2}$$

with clip parameter ϵ. Notably the critic constantly approximates the advantage in order to generate the most accurate loss. Therefore the reward only influences the learning of the critic directly. The encoder does not need to estimate the future reward, it operates on a time step by time step basis. Therefore we directly multiply the reward r with the DG output f_{enc}:

$$L_{enc} = r_{enc} * f_{enc}. \tag{3}$$

In general, there are a couple of constraints on the model, which we wanted it to fulfill. The model should use the resources it has, utilizing all sequences at some point. Currently there is only one environment and task, eliminating the need to allocate resources to different paradigms. Secondly since there are a limited amount of sequences, these ideally should not be active at the exact same time, but their place fields should cover the entire map. Internally this is translated to a temporal distance between activation of sequences. The current solution is to implemented several additional losses. The decoder was punished when actions lead to multi-activations of sequences, addressing the second constraint. The encoder has three additional losses:

1. Multi-activation loss: A punishment for sequences that receive input at the same time step, which targets the second constraint as well.
2. Unused-sequence loss: An encouragement for sequences that receive input, when they were previously completely inactive. Both this and the next loss target the general inclusion of currently unused sequences as put forth by the first constraint.
3. Batch loss: An encouragement for sequences which never get activated over a mini-batch. This is applied to all time steps in the corresponding mini-batch.

Every additional loss only affects the sequences which generated the loss, others are not affected. The idea behind these losses is shape the DG projection to utilize all sequences without multi-activation of sequences.

4 Results

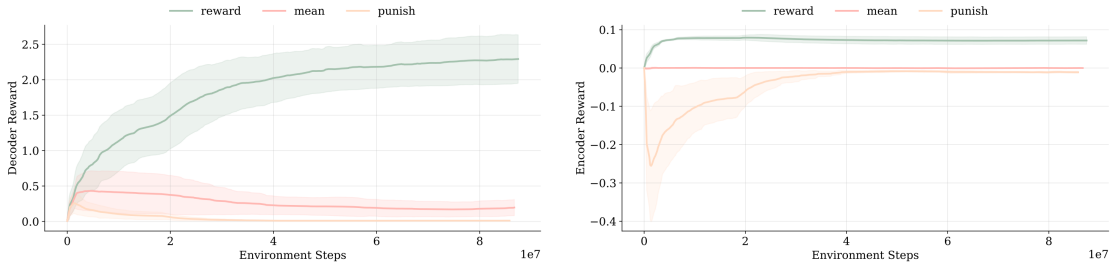

Figure 3: The evolution of reward over time, expressed in environment steps. Both encoder and decoder rewards converge quickly (under six hours training time). Here all intercept conditions are taken into account, differences are plotted as the standard deviation.

In general all runs converged quickly (under four hours computation time, see fig. 3), with learned behaviour and DG activation patterns. Generally a faster convergence to a stable amount of reward is seen for the encoder rewards (independent of reward calculation method), as well as the decoder reward. In the following we will first look at the different ways of encoder reward shaping. Additionally the effect of the high activation threshold is checked. For the best performing model, the learning is looked into in more detail as well as the effect of the additional losses gets controlled. Lastly a short investigation of transfer learning is performed. It has to be noted, that due to the exploratory nature of this project, computation time was mostly spent on exploring different hyperparameters. Thus, if not stated otherwise, each condition consists of runs with two different seeds (8,99). Unfortunately this makes comparative tests between conditions not possible. All results and claims and interpretations based on these results are purely qualitative. To check wether convergence of runs is vastly different for different seeds, one compare run, based on 10 different seeds was performed (see appendix). All runs converged to the same reward levels and displayed similar behaviour, as well as place field emergence.

4.1 Encoder Reward Shaping

The three different ways of encoder reward shaping all lead to different behaviour and firing maps of the DG projection cells (example in fig. 4). This is analyzed in terms of the two main components of interest: The DG projection to CA3 core and a behaviour analysis. Since purely positive or negative rewards change the overall activity levels of the projection, additionally we investigate different intercept values, which control the sparsity of projection activation. The activity maps calculated for the subsequent analyses were calculated using a spatial binning equalling the length of an environment square, if not stated differently.

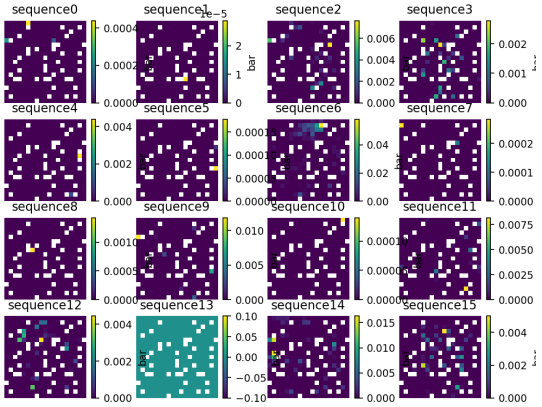

Figure 4: Example DG activation maps. Data is normalized by occupancy, white squares are obstacles.

DG Projection Analysis

Looking at the emergence of place fields in the projection from DG to CA3, an intuitive metric is the correlation between firing patterns for different sequences. Results are plotted in fig. 5, the

following paragraphs go through the individual panels.

Participation Ratio: We calculate the participation ratio of the correlation matrix, which is plotted in fig. 5. The participation ratio shows along how many dimensions the neural firing pattern is spread. Here a higher value means lower correlation between sequence activation. Generally it is visible that all conditions reach higher participation ratios than the random control. However there is no indication of any difference between the different reward calculation methods.

Gini Per Sequence: To investigate wether the activations of sequences are highly localized, we compute the Gini coefficient [20]. Adapted from the field of economics, it measures the area enveloped by the cumulative sum distribution of a variable, compared to a uniformly distributed variable. The ratio applied to activation maps measures how concentrated a potential place field is. If all firing is highly localized, the coefficient tends towards 1, for no spatial preference, a coefficient of 0 is expected.

Here fig. 5 the encouragement method exhibits a lower Gini coefficient, while both punishment and mean method show spatially highly concentrated activation patterns and therefore a high Gini coefficient.

Entropy Per Sequence: The same pattern emerges, when instead of the Gini coefficient, the entropy over activity maps is used. The analysis of the Gini coefficient and activation entropy was done on a per sequence basis, resulting in more data points than the participation ratio, which is only calculable over an entire trial. Note that entropy values of zero in both the Gini coefficient and entropy analysis represent sequences, which never got any input over the course of analysis. For the random control there were only three active sequences, the others never get high enough input to get activated (The intercept adaption only unfolds its effect, during training and the random control was never trained).

Coverage: Ideally place fields are spaced apart covering the entire environment roughly equally. To validate this, the coverage is calculated as the inverse OR 1- the Gini coefficient over the summed activity patterns of all sequences (The binning was set to a 4×4 grid: there is a theoretical bin for every sequence, which would lead to the highest coverage value). When place fields are spaced apart and activity does not overlap, the resulting value is high. This is especially the case for the reward method, although all methods show a great increase compared to the random control.

Activity Ratio: It is important to keep in mind that the overall activity levels between methods vary greatly. We measured this by percentage of sequence activations. Activations are summed over neurons and then the mean over all time steps is computed. The highest activity levels originate in the reward method, followed by less activity in the mean method. By far the sparsest, even compared to the random control, activity is seen in the punish method.

Behaviour Analysis

The effects of the agents behaviour are quantitatively calculated by the entropy of individual trajectories seen in Panel C of fig. 5, a qualitative view of individual trajectories is displayed in fig. 6 for the punishment method. The data is separated into runs of 900 frames length (Each frame consists of 8 environment frames, as 7 are skipped during training and analysis.) and then coarsely binned into four spatial quadrants. The idea is that runs, which cover the map equally have a higher entropy. However the exact trajectories vary a lot in space and entropy over a finer binning does not capture the behaviour coverage of the entire map, but instead how equally visited the covered places are.

The punishment method shows the highest entropy with the least spread across runs. Individual trajectories are also shown in the appendix for all methods, where an intuition for the behaviour can be developed. The punishment runs show a general exploration of the entire environment in every run, regardless of the starting position. Especially the mean reward method shows a lot of circular behaviour, which keeps individual runs confined to smaller areas of

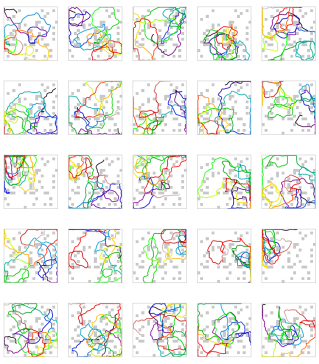

Figure 6: 25 example trajectories for the punishment method at intercept value of 2.43. Starting point is black.

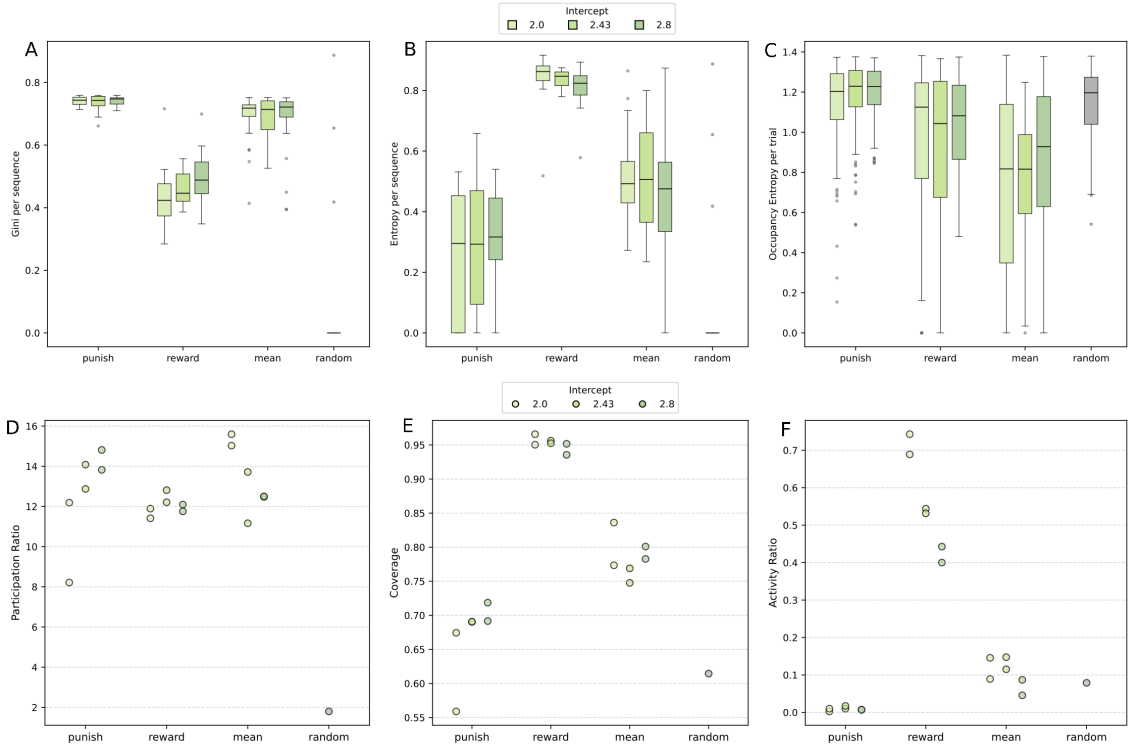

Figure 5: All results (except random trial) include data from runs with two different seeds. **A**: Gini coefficient calculated per sequence input (n=32). **B**: Entropy of activity patterns per sequence input (n=32). **C**: Entropy of the occupancy of single runs. Binning divides the map into four quadrants. **D**: The participation ratio calculated over the correlation matrix (n=112). **E**: Coverage measurement as 1 minus the Gini coefficient over the summed activity of sequence inputs. **F**: The fraction of sequences receiving input.

the environment. However the increased circular behaviour in the encouragement method seems to decrease the chance of the agent being stuck in corners in tricky parts of the environment. This can be quantified through the entropy of the occupancy over the entire 50000 analysis frames. When an agent repeatedly gets stuck in specific places, the local occupancy increases, which leads to a decrease in entropy. Ideally the agent exhibits high occupancy entropy, which is the case for both encouragement and the mean methods? So while individual runs might cover the environment better, there seems to be a higher chance of the agent getting temporarily stuck.

Batch Normalization Intercept

Since the different methods exhibit different activation levels for the DG projection, we wanted to control for this by changing the activation levels set by the intercept effect of the batch normalization and ReLu-activation (Different shades in fig. 5). For pure external reward training, the intercept level was set to be 2.43, letting only activations pass through, which exceed 2.43σ standard deviation (std) of the running estimate of mean and std. Note that this happens on a per-neuron-level. We adjusted the intercept to both 2 and 2.8. While all results of the different trials can be seen in the corresponding figures of the previous analyses, there are three results that are worth mentioning, apart from the general, expected change in activation levels. Firstly the entropy of DG projection activity patterns seems to decrease with higher intercept for the reward method, which consequently is reflected in an increase in the corresponding Gini coefficient. Secondly the seems to be an increase in behavioural occupation entropy, when calculated for small bin sizes (maybe leave out?). Otherwise the effect of intercept does not seem to affect the calculated metrics. Thirdly for the punishment method, which already has the lowest activity levels, there

are more completely inactive sequences for a lower intercept level. This seems counterintuitive, since lower intercept normally equals generally higher activity levels.

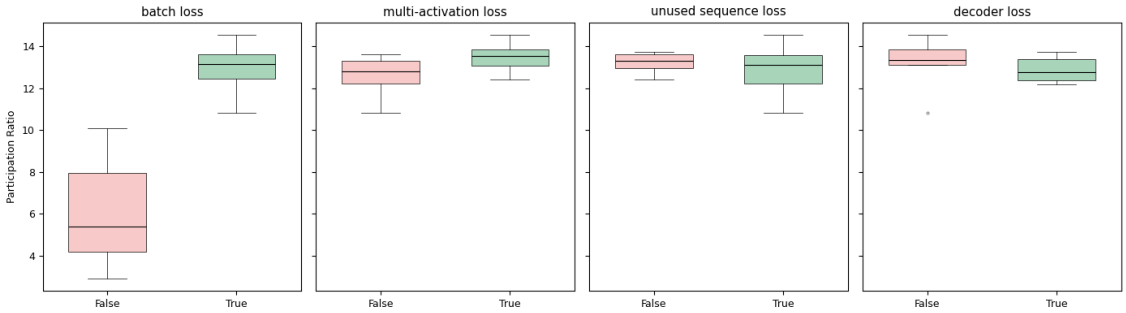

Figure 7: An example analysis (more are displayed in the Appendix) for the participation ratio. Every loss combination was run with two different seeds and then pooled over individual losses. Since the batch loss shows the most effect, it was set to True for the analysis of the rest of the losses.

4.2 Additional Losses

The additional incorporated losses all have a intuitive reasoning supporting them, as described above. However the practical effects of these losses needs to be understood as well. Therefore we ran all combinations for the four implemented additional losses. Due to computation time limits this was restricted to the punishment method, which showed the most promising results in the above results, see (section 5 for discussion. The same analyses as for encoder reward shaping were performed. It has to be noted that in fig different data points represent different loss combinations with only two different seeds. A detailed plot for the participation ratio and place field coverage is shown in the appendix with each combination of losses explicitly stated. The most pronounced effect can be observed for the batch loss, which drastically increases participation ratio as well as decreasing place field entropy and increasing the place field Gini coefficient, all pointing to a better performance. However place field coverage of the map notably decreases with the batch loss. Since the effect in all metrics is pronounced, for analysis of the other losses, the batch loss was always set active. Even then the only minor difference in performance seems to be the decoder loss, showing the opposite effects of the batch loss in a reduced magnitude.

4.3 Transfer Learning

In a short-lived effort the effects of the intrinsic reward training on a subsequent external reward paradigm was investigated. The aim of transfer learning was to reach a specified goal location, ideally in the fastest way possible. Constant reward was given on reaching the goal location. However in all cases training time to reach comparable performance was at most equal and often times greater than pure external reward training. Trials included all aforementioned pre-training methods (Additional Losses always active), with different intercept levels. Since the agent constantly evaluates the expected future reward, the last linear critic readout layer was reset before transfer. To check for potential interference in some trials the entire decoder structure was reset. Additionally we tried fixing the encoder weights for transfer learning. Due to time constraints, there were no further efforts to analyze these trials in more detail and potentially improve the transfer learning. In the discussion section 5, a few potential reasons for failed transfer learning are given, as well as potential further research ideas in this domain.

5 Discussion

The interpretation of results has two components: place field generation, as well as a behavioural aspect. While all methods show better performance than a random agent, the results indicate that the pure punishment method shows the most promising place fields. This is true for both the tuning, as well as the coverage over the entire environment. However the latter might be coupled to the way different agents explore the environment. The punished agent generally explores more quadrants of the map in each run, which might lead to a better neural representation of the entire map. The circular behaviour observed in the other methods could lead to an over-representation of specific areas. In addition to the interplay between behaviour and neural activity, a potentially great factor in the emergence of place fields is the additional sparsification of input through a constant punishment. This generally decreases the activity, leading to the selection of only the most pronounced landmarks.
However there seems to be a lot of noise in the localized activation, where individual sequences get activated at multiple, different locations, with areas of no activation in between. This seems to be true for all methods, but is most notable in the punished agent. It is not entirely clear if this is due to generally lower activity level, but artificially increasing the intercept level to achieve higher activity does not generally change neither sequence activation patterns, nor behavioural outcomes.
Even though there are intuitive reasons for the extra losses added in training, not all losses are reflected in better performance in the key metrics analyzed here. However the batch loss improves performance significantly. The reason is that the metrics generally reflect in parts how many sequences are active, which is the key objective this loss addresses. So naturally the participation ratio and place field coverage improve greatly with more sequences. But place fields seem to also become more localized as indicated by the Gini coefficient and entropy of place fields when the batch loss is active. When looking at this in the light of an increase in coverage for active batch loss, one explanation is that more active sequences require each sequence to cover less physical space in the environment, allowing the place field of each neuron to be better tuned. Generally it seems that all losses do not negatively influence the performance in the investigated metrics and the general reasoning behind them remains valid. However the exact effects of every individual loss would need to be investigated in more detail.

Future additions

Generally it might be beneficial to find an intuitive way of incorporating the losses into the individual encoder and decoder rewards. Tailoring these to increase performance would lead to a more coherent model, with less artificially set boundaries and a pure internal reward system.
The training of encoder and decoder currently happens at the same. Potentially decoupling them in either an alternating training scheme or a pre-training of the encoder with fixed decoder weights might lead to interesting results.
Even though transfer learning did not yield promising results, this might be due to the vastly different nature of the two training paradigms. Aligning an external reward task to the internal map learned in pre-training might lead to interesting insights on adaption and usage of spatial maps, learning through pure exploration. Additionally individual parts (such as the additional losses) of the present model might improve performance on external reward paradigms if applied during normal training.
This report is only able to show a glimpse into the potential of internal reward learning and there is a lot of potential for further research.

6 Conclusion

The present report shows that hippocampal-like spatial representations with navigational behaviour emerges from a purely intrinsic reward. A key factor is the suppression of activity in the projection from the dentate gyrus to the CA3-like sequence core, as well as a general encouragement to utilize all sequences.

Acknowledgments

Primarily I want to thank my supervisors Christian Leibold and Xiao-Xiong Lin for providing me with the opportunity to conduct this research project. Especially Xiao-Xiong always had an open ear for my problems and many fruitful discussions were held during the three months.
Additionally, the author would like to thank the state of Baden-Württemberg for its support of bwHPC and the German Research Foundation (DFG) for funding under "Project number 455622343" (bwForCluster NEMO 2).

References

[1] Charles Beattie et al. *DeepMind Lab*. Dec. 2016. DOI: 10.48550/arXiv.1612.03801. arXiv: 1612.03801 [cs]. (Visited on 11/17/2025).

[2] Lukas Biewald. "Experiment Tracking with Weights and Biases". In: (2020). (Visited on 11/17/2025).

[3] Lancelot Da Costa et al. "Reward Maximization Through Discrete Active Inference". In: *Neural Computation* 35.5 (Apr. 2023), pp. 807–852. ISSN: 0899-7667, 1530-888X. DOI: 10.1162/neco_a_01574. (Visited on 11/17/2025).

[4] Arne D. Ekstrom et al. "Cellular Networks Underlying Human Spatial Navigation". In: *Nature* 425.6954 (Sept. 2003), pp. 184–188. ISSN: 1476-4687. DOI: 10.1038/nature01964. (Visited on 11/17/2025).

[5] David J. Foster and Matthew A. Wilson. "Hippocampal Theta Sequences". In: *Hippocampus* 17.11 (Nov. 2007), pp. 1093–1099. ISSN: 1050-9631, 1098-1063. DOI: 10.1002/hipo.20345. (Visited on 11/17/2025).

[6] Tobias Jung, Daniel Polani, and Peter Stone. "Empowerment for Continuous Agent—Environment Systems". In: *Adaptive Behavior* 19.1 (Feb. 2011), pp. 16–39. ISSN: 1059-7123, 1741-2633. DOI: 10.1177/1059712310392389. (Visited on 11/17/2025).

[7] Kenneth Kay et al. "Constant Sub-second Cycling between Representations of Possible Futures in the Hippocampus". In: *Cell* 180.3 (Feb. 2020), 552–567.e25. ISSN: 00928674. DOI: 10.1016/j.cell.2020.01.014. (Visited on 11/17/2025).

[8] Christian Leibold. "A Model for Navigation in Unknown Environments Based on a Reservoir of Hippocampal Sequences". In: *Neural Networks* 124 (Apr. 2020), pp. 328–342. ISSN: 08936080. DOI: 10.1016/j.neunet.2020.01.014. (Visited on 04/19/2025).

[9] Xiao-Xiong Lin, Yuk Hoi Yiu, and Christian Leibold. *Egocentric Visual Navigation through Hippocampal Sequences*. Oct. 2025. DOI: 10.48550/arXiv.2510.09951. arXiv: 2510.09951 [q-bio]. (Visited on 11/17/2025).

[10] Volodymyr Mnih et al. *Asynchronous Methods for Deep Reinforcement Learning*. June 2016. DOI: 10.48550/arXiv.1602.01783. arXiv: 1602.01783 [cs]. (Visited on 08/05/2025).

[11] Rubén Moreno-Bote and Jorge Ramírez-Ruiz. "Empowerment, Free Energy Principle and Maximum Occupancy Principle Compared". In: ().

[12] Lynn Nadel and Sara Aronowitz, eds. *Space, Time, and Memory*. 1st ed. Oxford University PressOxford, May 2025. ISBN: 978-0-19-288254-7 978-0-19-197681-0. DOI: 10.1093/oso/9780192882547.001.0001. (Visited on 11/17/2025).

[13] Thomas C. Neylan. "Memory and the Medial Temporal Lobe: Patient H. M." In: *JNP* 12.1 (Feb. 2000), pp. 103–103. ISSN: 0895-0172, 1545-7222. DOI: 10.1176/jnp.12.1.103. (Visited on 11/17/2025).

[14] J. O'Keefe and J. Dostrovsky. "The Hippocampus as a Spatial Map. Preliminary Evidence from Unit Activity in the Freely-Moving Rat". In: *Brain Research* 34.1 (Nov. 1971), pp. 171–175. ISSN: 00068993. DOI: 10.1016/0006-8993(71)90358-1. (Visited on 11/17/2025).

[15] Aleksei Petrenko et al. *Sample Factory: Egocentric 3D Control from Pixels at 100000 FPS with Asynchronous Reinforcement Learning*. June 2020. DOI: 10.48550/arXiv.2006.11751. arXiv: 2006.11751 [cs]. (Visited on 04/30/2025).

[16] Jorge Ramírez-Ruiz et al. "Complex Behavior from Intrinsic Motivation to Occupy Future Action-State Path Space". In: *Nat Commun* 15.1 (July 2024), p. 6368. ISSN: 2041-1723. DOI: 10.1038/s41467-024-49711-1. (Visited on 11/17/2025).

[17] *Sample Factory Fork*. https://github.com/JONNeKK/sample-factory/tree/feature/custom_learner. (Visited on 11/17/2025).

[18] John Schulman et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. Oct. 2018. DOI: 10.48550/arXiv.1506.02438. arXiv: 1506.02438 [cs]. (Visited on 04/30/2025).

[19] John Schulman et al. *Proximal Policy Optimization Algorithms*. Aug. 2017. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs]. (Visited on 04/30/2025).

[20] "Statistical Inference Via Bootstrapping for Measures of Inequality". In: *JOURNAL OF APPLIED ECONOMETRICS* 12 (1997).

Appendix

Changes to the code base

This section is dedicated as a short summary of the implementation details needed to make the SampleFactory codebase work with this project. The general framework tries to achieve a high level of separation between components, while trying to keep communication between them to a minimum. In all additions, this philosophy was followed in the best possible way, while keeping the workload of implementation to a minimum. The injection of internal reward happens entirely in the Learner class. It does not rely on any information of the sampling module and can be directly executed prior to the training step. This eliminates the need for transfer of internal rewards between the CPU and GPU. To accommodate these calculations a possibility of minimal injection into the existing training process was implemented. Akin to how custom parts of the agents model can be injected before each training session starts, it is now possible to add custom learning classes. This creates the opportunity to change the entire training procedure, including the addition of new loss functions, access to the gradients and optimizers, as well as training batch preparation and loading of arbitrary model parameters.
The implementation happened mainly in the first half of the project, which lay the necessary groundwork for the successful realization of this project. The source code can be found on Github [17] in the branch 'feature/custom_learner'. In the future there might be changes to this code, since the repository is still used in active development.

Additional figures

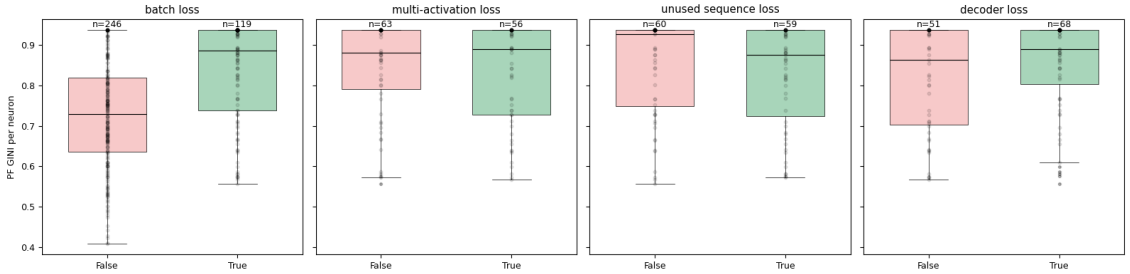

Figure 8: Gini coefficient per neuron for different loss conditions, pooled over all other conditions, with the exception of batch loss being set to True for all other conditions.

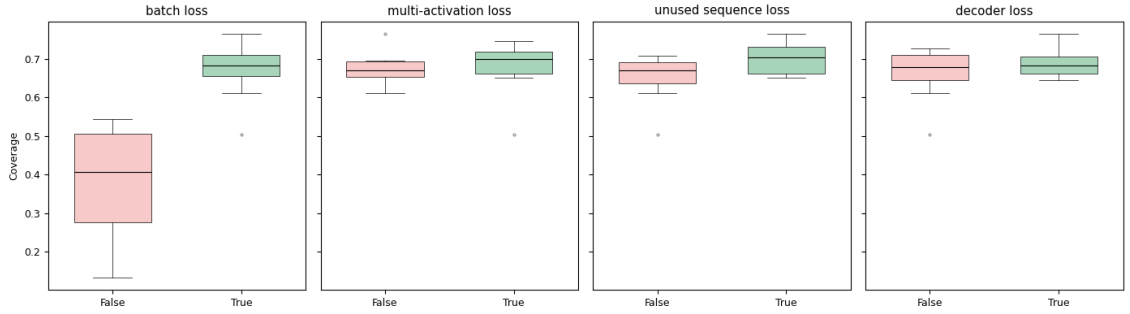

Figure 9: Coverage of the map for different loss conditions, pooled over all other conditions, with the exception of batch loss being set to True for all other condtions.

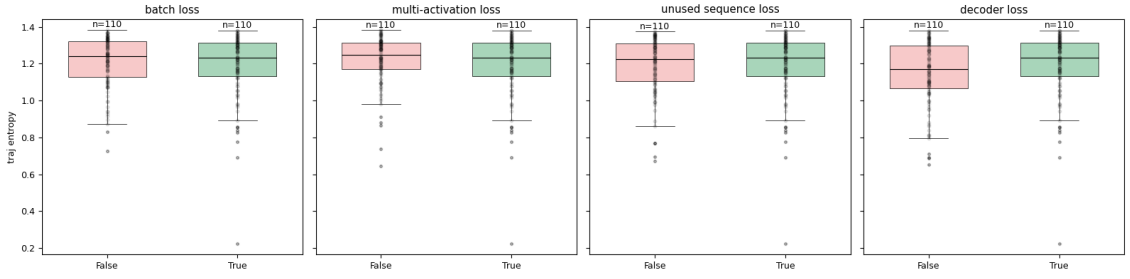

Figure 10: Plotting of the trajectory entropy for different loss conditions. If one is varied, all others are set True.